

Pontifícia Universidade Católica de Campinas
Escola Politécnica



Arquitetura de um Sistema Gerenciador de Banco de Dados

Prof Dra Renata Antonia Tadeu Arantes
E-mail: renata.arantes@puc-campinas.edu.br

Arquitetura

- Interna do SGBD (camada externa , conceitual , interna , componentes)
- De Implantação (centralizado, cliente-servidor, distribuído , nuvem)

Onde estão os Bancos de Dados?

NETFLIX

SPOTIFY

IFOOD

INSTAGRAM

Exemplos do dia a dia:

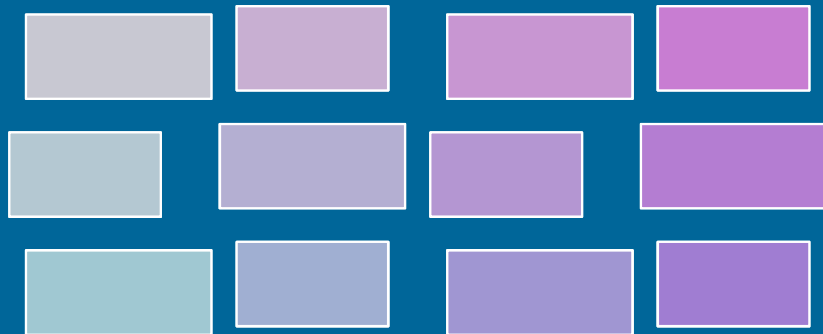
- Redes sociais (Instagram)
- Streaming (Netflix, Spotify)
- E-commerce (iFood)

Todos têm em comum: BANCOS DE DADOS (usuários, itens, transações, preferências, histórico).

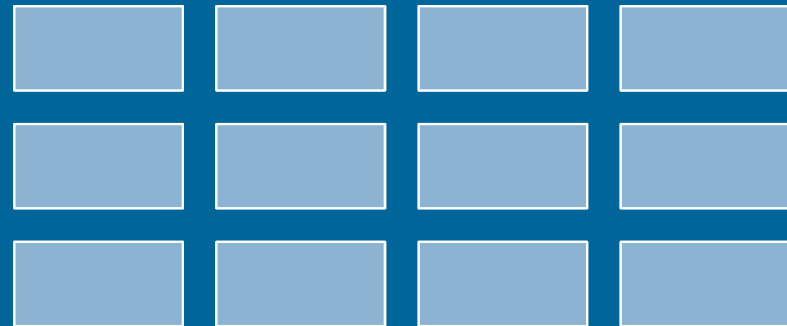
O que é um SGBD?

Sistema Gerenciador de Banco de Dados (SGBD) é um software que permite criar, gerenciar e manipular dados.

Sem SGBD: arquivos soltos, duplicados, inconsistentes



Com SGBD: organização, integridade, segurança



Metáfora: Biblioteca

- Sem bibliotecário x Com bibliotecário (catalogação, controle de empréstimos, integridade)

Arquitetura de um SGBD — 3 Níveis

Nível Externo (Visões do Usuário)

Relatórios, telas de app, permissões por perfil

Nível Conceitual (Modelo Lógico)

Entidades, relacionamentos, restrições

Nível Interno (Armazenamento Físico)

Arquivos, índices, páginas, buffers

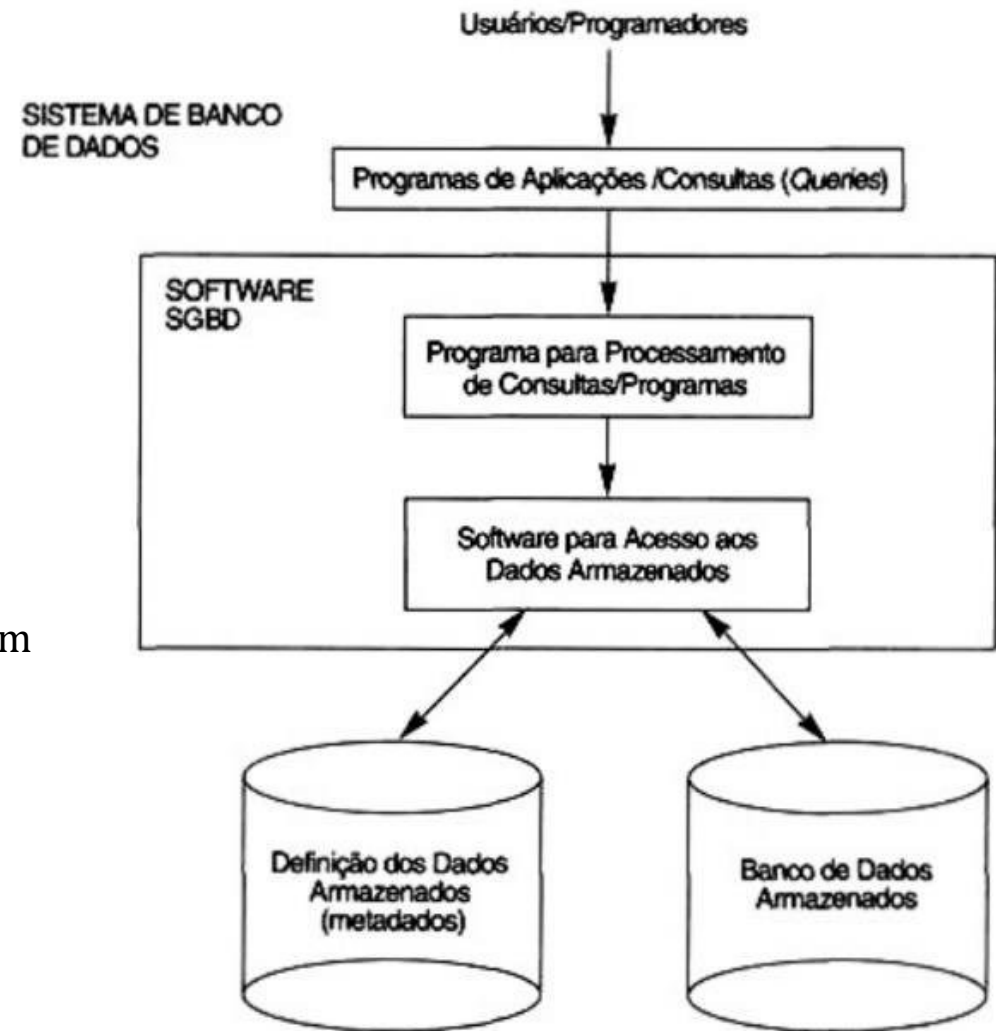
Exemplo Netflix:

- Externo (Minha Lista)
- Conceitual (Usuário–Assinatura–Filme)
- Interno (tabelas nos servidores)

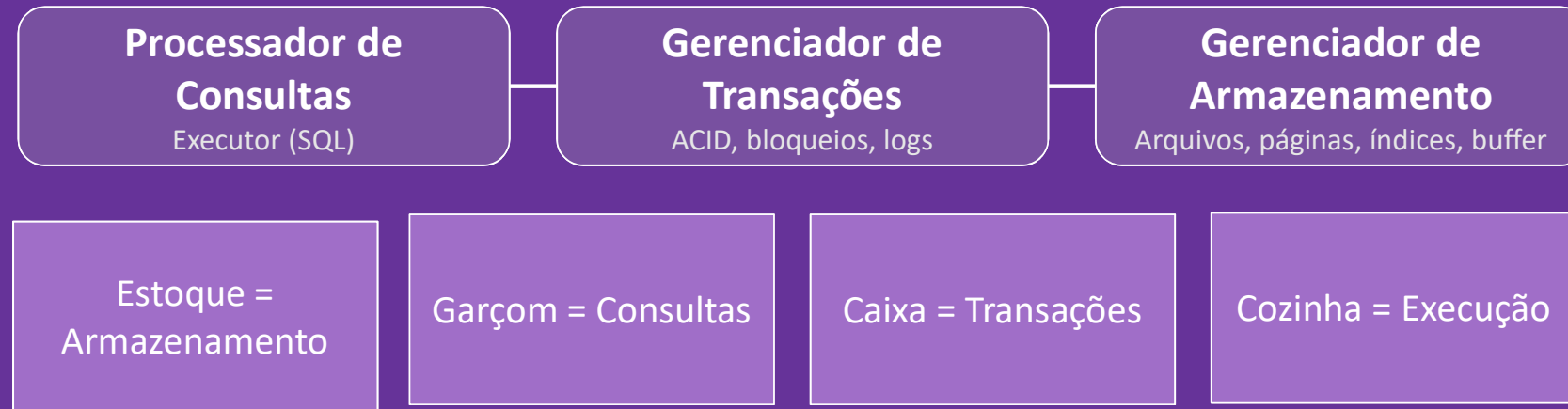
Esse diagrama mostra como funciona um SGBD de forma simplificada.

1. Usuários/Programadores → não acessam os dados diretamente, usam aplicações ou SQL.
2. Programas de Aplicação/Consultas → sistemas que enviam consultas ao banco.
3. Software SGBD →
 - Processamento de Consultas/Programas: interpreta o SQL, otimiza e organiza a execução.
 - Acesso aos Dados Armazenados: busca dados físicos em disco usando índices e buffers.
4. Armazenamento →
 - Metadados: descrevem a estrutura (tabelas, colunas, restrições).
 - Banco de Dados Armazenados: dados reais (clientes, produtos, transações).

Resumindo: usuário → consulta → processador → armazenamento → resposta.



Componentes do SGBD (Visão Simplificada)



Gerenciador de Armazenamento 📦

- Processador de Consultas (SQL) 🧑🔍
- Gerenciador de Transações 💰
- Gerenciador de Buffer/Cache 🗄️

Metáfora: Restaurante

- Estoque = armazenamento
- Garçom = consultas
- Caixa = transações
- Cozinha = processamento

O SGBD possui funções internas bem definidas:

1. Processador de Consultas → entende o SQL, otimiza e executa. Como o garçom, que entende o pedido do cliente.
2. Gerenciador de Transações → garante as regras, consistência, isolamento, durabilidade. É o caixa, que controla e registra corretamente cada operação.
3. Gerenciador de Armazenamento → administra arquivos, páginas, índices e buffers no disco. É o estoque, onde ficam os ingredientes (dados).
4. Buffer/Cache → mantém dados em memória para acesso mais rápido. É a cozinha, que agiliza o preparo com os ingredientes à mão.

Resumindo: usuário faz a consulta, o garçom entende, o caixa garante consistência, a cozinha processa e o estoque fornece os dados.

Exemplo Rápido: Spotify

Nível Externo

Playlist do usuário, curtidas,
recomendações

Nível Conceitual

Entidades: Usuário, Playlist,
Música
Relacionamentos: Usuário–
Playlist, Playlist–Música

Nível Interno

Tabelas/índices em disco, cache
em memória, partições

Arquiteturas de Implantação de Bancos de Dados

Centralizado • Cliente-Servidor • Distribuído •
Nuvem

Arquitetura Interna x Arquitetura de Implantação

Arquitetura Interna (SGBD)

Camadas: Externa, Conceitual, Interna

→ Como o banco organiza os dados por dentro

Arquitetura de Implantação

Centralizado, Cliente-Servidor, Distribuído, Nuvem

→ Como o banco é instalado e acessado

Arquitetura Centralizada

- Um único servidor concentra o banco de dados
- Todo o processamento ocorre no servidor
- Clientes apenas enviam comandos/resultados

✓ Simples, fácil de administrar

✗ Gargalo e ponto único de falha

Exemplo: mainframes antigos, sistemas legados locais

Arquitetura Centralizada

- **Servidor Central:** O coração do sistema, onde todo o banco de dados reside. Este servidor é responsável por armazenar, gerenciar e proteger todos os dados.
- **Clientes ou Terminais:** Usuários acessam o banco de dados através de terminais ou estações de trabalho que estão conectados ao servidor central, geralmente por uma rede local (LAN). Esses terminais não armazenam dados localmente; eles simplesmente enviam solicitações ao servidor central.

Arquitetura Centralizada

- **Funcionamento**
- **Processamento de Requisições:** Quando um usuário precisa de dados, ele envia uma requisição ao servidor central. O servidor processa essa requisição, acessa o banco de dados, e envia os dados de volta ao terminal do usuário.
- **Execução de Transações:** Todas as transações (como inserções, atualizações, exclusões de dados) são gerenciadas pelo servidor central. Isso garante que todas as operações sejam executadas de forma consistente e segura.
- **Armazenamento e Backup:** Todo o armazenamento de dados e backups são gerenciados de maneira centralizada, facilitando a administração e a recuperação de dados.

Arquitetura Centralizada

- **Vantagens**
- **Simplicidade de Gerenciamento:** Com todos os dados centralizados em um único local, a administração do banco de dados (como backup, segurança, e manutenção) é mais simples e direta.
- **Controle Centralizado:** As políticas de segurança e controle de acesso são mais fáceis de implementar e monitorar, pois todas as operações passam pelo servidor central.
- **Menor Custo de Hardware:** Como os terminais dos usuários não precisam de grande capacidade de processamento ou armazenamento, o custo do hardware pode ser reduzido.

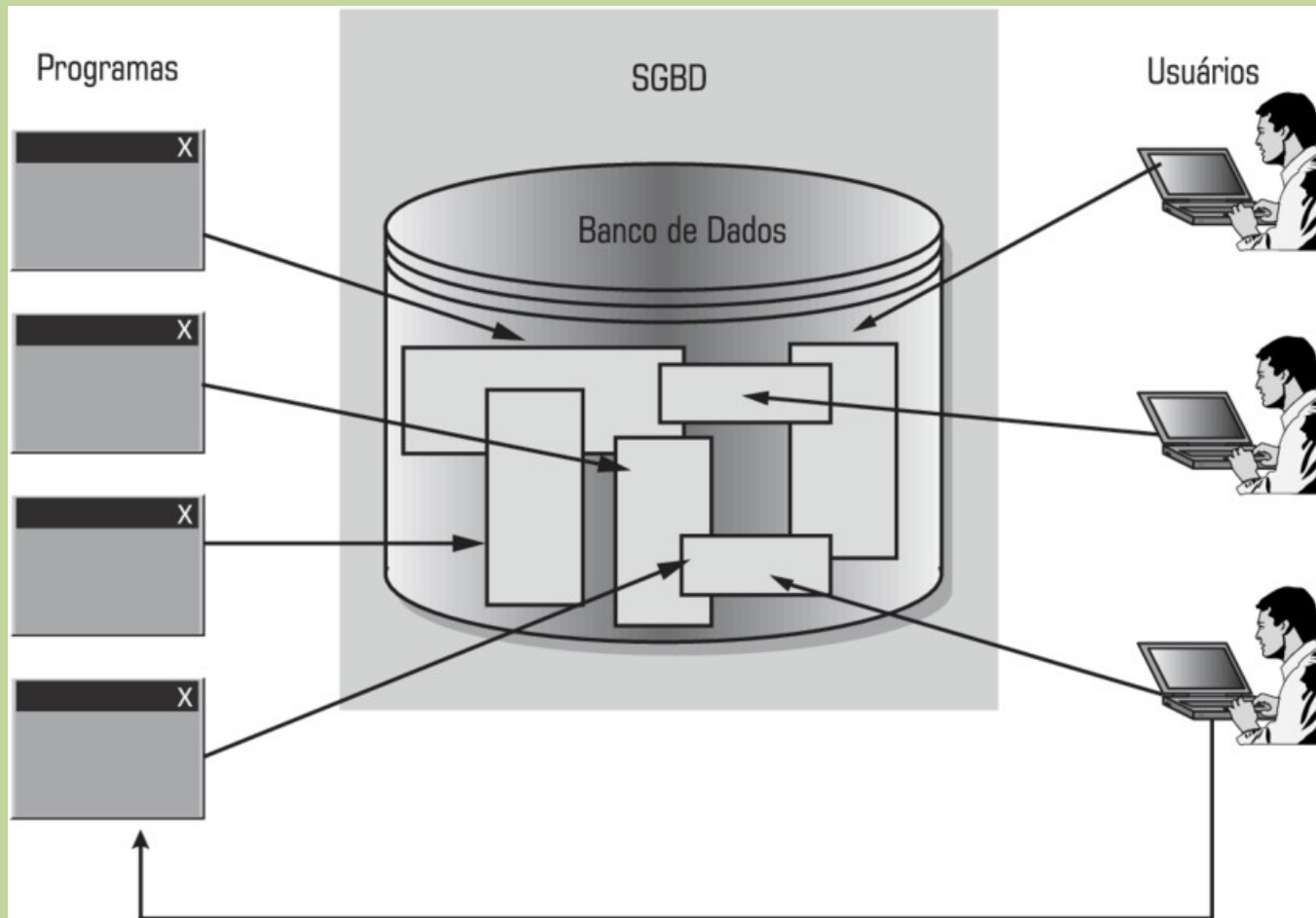
Arquitetura Centralizada

- **Desvantagens**
- **Ponto Único de Falha:** Se o servidor central falhar, todo o sistema fica indisponível. Isso pode resultar em tempo de inatividade significativo até que o problema seja resolvido.
- **Escalabilidade Limitada:** À medida que o número de usuários ou a quantidade de dados cresce, o servidor central pode se tornar um gargalo, levando a problemas de desempenho.
- **Dependência de Rede:** O desempenho do sistema depende da qualidade e velocidade da conexão de rede entre os terminais e o servidor central.

Arquitetura Centralizada

- **Exemplos**
- **Sistemas Bancários Tradicionais:** Muitos sistemas bancários mais antigos eram baseados em arquiteturas centralizadas, com todas as transações processadas em um data center central.
- **Sistemas de Gerenciamento de Informação (MIS):** Algumas pequenas empresas usam um único servidor para centralizar todas as suas operações, como contabilidade, RH e CRM.
- A arquitetura centralizada é ideal para ambientes controlados onde a simplicidade e o controle centralizado são prioridades, mas pode se tornar um desafio à medida que as necessidades de escalabilidade e disponibilidade crescem.

Servidor Central



Arquitetura Cliente-Servidor

- Aplicações (clientes) fazem parte do processamento (interface, lógica)
- Servidor central guarda e processa o banco de dados

✓ Mais eficiente que centralizado

✗ Dependência do servidor

Exemplo: aplicações corporativas com Oracle, MySQL, PostgreSQL

Arquitetura Cliente-Servidor

- **Estrutura Básica**
- **Servidor:** O servidor é o coração do sistema e geralmente é um computador robusto que hospeda o banco de dados. Ele é responsável por armazenar os dados, gerenciar a lógica de negócios e executar consultas e transações.
- **Cliente:** O cliente é um dispositivo de usuário final, como um PC, laptop ou dispositivo móvel. Ele executa a interface do usuário e algumas das lógicas da aplicação. O cliente envia requisições ao servidor e recebe respostas, geralmente na forma de dados processados.

Arquitetura Cliente-Servidor

- **Funcionamento**
- **Conexão e Comunicação:** O cliente se conecta ao servidor por meio de uma rede, como uma LAN (Local Area Network) ou a internet. A comunicação entre cliente e servidor é feita por meio de protocolos de rede, como TCP/IP.
- **Processamento Distribuído:** O cliente é responsável por capturar a entrada do usuário, exibir os resultados e executar parte do processamento, enquanto o servidor lida com o armazenamento, processamento e gerenciamento dos dados.
- **Envio de Requisições:** Quando um usuário faz uma operação, como consultar um banco de dados, o cliente envia uma requisição ao servidor. O servidor processa a requisição, executa a consulta no banco de dados e retorna os resultados ao cliente.
- **Execução de Transações:** As transações, que podem incluir operações de leitura e escrita no banco de dados, são gerenciadas pelo servidor para garantir consistência, integridade e segurança.

Arquitetura Cliente-Servidor

- **Vantagens**
- **Escalabilidade:** A arquitetura cliente-servidor pode ser facilmente escalada, adicionando mais servidores para suportar mais clientes ou distribuindo a carga entre vários servidores.
- **Flexibilidade:** Diferentes clientes (como aplicativos desktop, web ou móveis) podem interagir com o mesmo servidor, proporcionando flexibilidade no desenvolvimento de aplicações.
- **Melhor Desempenho:** Como o processamento é dividido entre cliente e servidor, o sistema pode ser mais eficiente, especialmente em operações complexas.
- **Manutenção Centralizada:** O servidor centraliza o gerenciamento de dados e a lógica de negócios, facilitando a atualização e manutenção do sistema.

Arquitetura Cliente-Servidor

- **Desvantagens**
- **Complexidade de Configuração:** A configuração inicial do sistema cliente-servidor pode ser mais complexa, exigindo um bom entendimento de redes, servidores e segurança.
- **Dependência da Rede:** A performance do sistema depende da qualidade e velocidade da conexão de rede. Em redes lentas ou instáveis, a comunicação entre cliente e servidor pode ser comprometida.
- **Custos:** Manter servidores robustos e seguros pode ser caro, especialmente em grandes sistemas com muitos usuários.

Arquitetura Cliente-Servidor

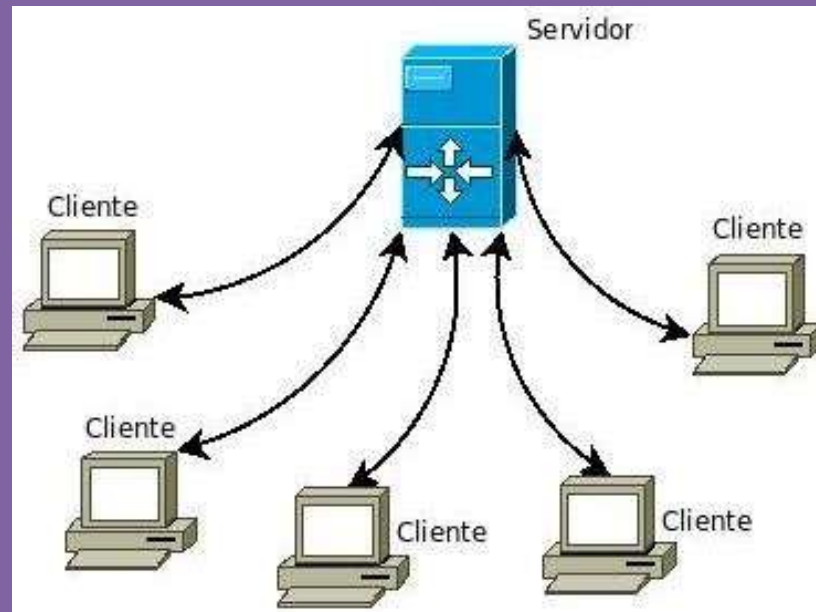
- **Exemplos**
- **Aplicações Web:** Um exemplo clássico é um site de e-commerce. O navegador do usuário (cliente) envia uma solicitação ao servidor para exibir uma página de produto. O servidor processa a solicitação, consulta o banco de dados para obter informações do produto e envia os dados de volta ao navegador para exibição.
- **Sistemas de Banco de Dados:** MySQL e PostgreSQL são exemplos de servidores de banco de dados que seguem essa arquitetura, onde aplicações (clientes) se conectam ao banco de dados para armazenar e recuperar dados.

Arquitetura Cliente-Servidor

- **Cenário de Funcionamento**
- Imagine um sistema de reservas de passagens aéreas:
- **Cliente:** O usuário acessa um aplicativo de reservas no celular ou um site no computador e solicita a reserva de uma passagem.
- **Servidor:** O servidor processa a solicitação, consulta o banco de dados para verificar a disponibilidade de voos, e retorna os resultados ao cliente. Se o usuário decide reservar um voo, o servidor processa a transação e atualiza o banco de dados com as novas informações.
- Essa arquitetura é muito poderosa e é o fundamento de muitos sistemas de TI modernos, permitindo que diferentes tipos de dispositivos acessem e interajam com dados e serviços de maneira eficiente e escalável.

Cliente-Servidor

- Na arquitetura Cliente-Servidor, o cliente (front_end) executa as tarefas do aplicativo, ou seja, fornece a interface do usuário (tela, e processamento de entrada e saída). O servidor (back_end) executa as consultas no DBMS e retorna os resultados ao cliente.



Arquitetura Distribuída

- Dados divididos ou replicados em múltiplos servidores
- Usuário acessa como se fosse um único banco

✓ Escalabilidade, redundância

✗ Configuração e sincronização complexas

Exemplo: Google, Amazon, bancos financeiros

Arquitetura Distribuída

- **Estrutura Básica**
- **Servidores Distribuídos (Nós):** Os dados são armazenados em múltiplos servidores ou nós, que podem estar localizados em diferentes regiões geográficas. Esses nós trabalham juntos para fornecer uma visão unificada dos dados aos usuários.
- **Cliente:** Assim como na arquitetura cliente-servidor, os clientes enviam requisições ao sistema para acessar ou modificar os dados. No entanto, na arquitetura distribuída, as requisições podem ser atendidas por qualquer um dos servidores disponíveis, dependendo de onde os dados estão localizados.

Arquitetura Distribuída

- **Funcionamento**
- **Fragmentação de Dados (Sharding):** Os dados são divididos em fragmentos (ou shards), e cada fragmento é armazenado em um servidor diferente. Essa fragmentação pode ser baseada em diferentes critérios, como intervalos de chaves primárias, tipos de dados ou regiões geográficas.
- **Replicação de Dados:** Para garantir a disponibilidade e a resiliência, os dados podem ser replicados em múltiplos servidores. Isso significa que cópias dos dados são mantidas em vários locais, de forma que, se um servidor falhar, os dados ainda possam ser acessados de outro servidor.
- **Coordenação de Transações:** Quando uma operação envolve dados em múltiplos servidores, um protocolo de coordenação, como o protocolo de commit em duas fases (2PC), é usado para garantir que todas as partes da transação sejam concluídas com sucesso ou que a transação seja revertida se ocorrer algum erro.
- **Consistência e Disponibilidade:** Sistemas distribuídos enfrentam o desafio do trade-off entre consistência, disponibilidade e tolerância a partições. Dependendo do sistema, pode-se optar por uma maior consistência (todos os nós têm os mesmos dados), ou maior disponibilidade (os dados podem ser acessados mesmo que alguns nós estejam inacessíveis).

Arquitetura Distribuída

- **Vantagens**
- **Escalabilidade:** A arquitetura distribuída permite escalar horizontalmente, ou seja, adicionar mais servidores para lidar com o aumento de dados ou carga de trabalho. Isso é especialmente útil para grandes volumes de dados e aplicações de alto desempenho.
- **Resiliência:** Se um servidor falhar, os dados podem ser acessados a partir de outros servidores, garantindo alta disponibilidade e resistência a falhas.
- **Desempenho Localizado:** Ao armazenar os dados mais próximos dos usuários ou aplicações que os utilizam, é possível melhorar o desempenho, reduzindo a latência e o tempo de resposta.
- **Segurança dos Dados:** Réplicas em diferentes locais reduzem risco de perda total por acidente, desastre natural ou ataque.

Arquitetura Distribuída

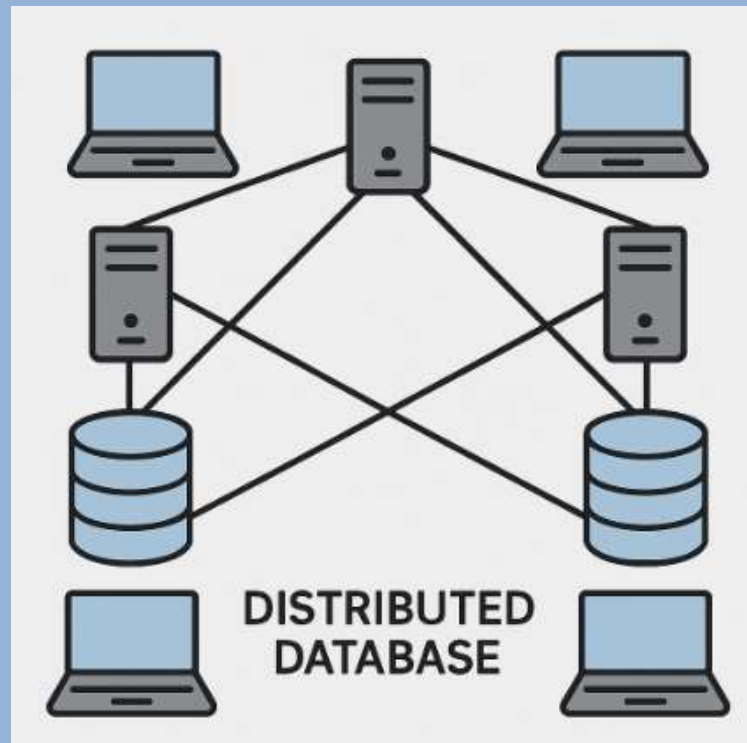
- **Desvantagens**
- **Complexidade de Gerenciamento:** Gerenciar um sistema distribuído é mais complexo, exigindo técnicas avançadas para lidar com a consistência, replicação, coordenação de transações e recuperação de falhas.
- **Problemas de Latência:** Embora os dados possam estar próximos dos usuários, a comunicação entre servidores distribuídos, especialmente em diferentes regiões geográficas, pode introduzir latência.
- **Consistência Eventual:** Em sistemas altamente distribuídos, pode-se optar por consistência eventual, onde os dados podem levar algum tempo para se propagar entre todos os servidores, o que pode causar discrepâncias temporárias.

Arquitetura Distribuída

- **Cenário de Funcionamento**
- Imagine um serviço de streaming global, como o Netflix:
- **Distribuição Geográfica:** Os filmes e séries são armazenados em data centers em diferentes partes do mundo. Quando um usuário na Europa faz streaming de um filme, ele é servido a partir de um servidor na Europa para minimizar a latência.
- **Replicação e Consistência:** Se um novo filme é adicionado ao catálogo, essa informação é propagada por todos os data centers para garantir que os usuários em qualquer região possam acessar o novo conteúdo.
- **Resiliência:** Se um data center na América do Norte falhar, os usuários podem ser redirecionados para um data center na América do Sul sem interrupção no serviço.
- Essa arquitetura é fundamental para sistemas modernos que exigem alta disponibilidade, grande capacidade de processamento e distribuição global. Ela permite que as empresas ofereçam serviços confiáveis e de alto desempenho, mesmo em escala global.

Estruturas das Plataformas

- Nesta arquitetura, a informação está distribuída em diversos servidores. Cada servidor atua como no sistema cliente-servidor, porém as consultas oriundas dos aplicativos são feitas para qualquer servidor indistintamente.



Bancos de Dados na Nuvem

- Banco de dados oferecido como serviço
- Acesso pela internet

✓ Escalabilidade, alta disponibilidade

✗ Dependência do provedor e custos

Exemplo: AWS RDS, Azure SQL, Google Firestore

Nuvem (cloud)

- **Estrutura Básica**
- **Provedor de Nuvem:** Os bancos de dados em nuvem são oferecidos por provedores de serviços de nuvem como Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), entre outros. Esses provedores fornecem a infraestrutura necessária para armazenar e gerenciar os dados.
- **Serviços Gerenciados:** Em vez de configurar e manter servidores físicos, as empresas utilizam serviços gerenciados de banco de dados, como Amazon RDS, Azure SQL Database ou Google Cloud SQL. Esses serviços cuidam da infraestrutura, incluindo provisionamento de hardware, instalação de software, backups, segurança e manutenção.

Nuvem (cloud)

- **Funcionamento**
- **Acesso via Internet:** Os bancos de dados em nuvem são acessados através da internet. Os usuários se conectam ao banco de dados usando credenciais apropriadas, independentemente da localização geográfica, o que facilita o acesso remoto e o trabalho em equipe distribuído.
- **Escalabilidade Automática:** Um dos principais benefícios dos bancos de dados em nuvem é a capacidade de escalar automaticamente. Se o volume de dados ou a carga de trabalho aumentar, o sistema pode adicionar mais recursos automaticamente, sem a necessidade de intervenção manual.
- **Pagamentos Sob Demanda:** As empresas pagam apenas pelos recursos que utilizam, como armazenamento, processamento e transferência de dados. Isso permite uma otimização dos custos, especialmente para startups e pequenas empresas que podem não ter recursos para investir em infraestrutura própria.
- **Backup e Recuperação Automatizados:** Os provedores de nuvem oferecem backup e recuperação de dados como parte do serviço. Isso inclui backups automáticos regulares e a capacidade de restaurar dados rapidamente em caso de falha.
- **Segurança e Conformidade:** Os provedores de nuvem implementam medidas de segurança avançadas, como criptografia de dados em trânsito e em repouso, controle de acesso rigoroso e conformidade com regulamentações internacionais.

Nuvem (cloud)

- **Vantagens**
- **Flexibilidade:** Os bancos de dados em nuvem podem ser rapidamente provisionados, redimensionados ou desativados, oferecendo flexibilidade para atender às necessidades de negócios dinâmicas.
- **Redução de Custos:** Elimina a necessidade de investir em hardware caro, instalações físicas, e pessoal de TI para gerenciar a infraestrutura. Os custos operacionais são reduzidos, e as empresas podem optar por pagar conforme o uso.
- **Alta Disponibilidade e Resiliência:** A infraestrutura de nuvem é projetada para oferecer alta disponibilidade e resiliência. Provedores como AWS, Azure e Google Cloud possuem data centers redundantes em várias regiões geográficas, garantindo que os serviços permaneçam operacionais mesmo em caso de falhas.
- **Atualizações e Manutenção Automáticas:** O provedor de nuvem cuida das atualizações de software, patches de segurança e outras tarefas de manutenção, liberando as equipes de TI para focar em atividades estratégicas.

Nuvem (cloud)

- **Desvantagens**
- **Dependência de Internet:** Como os bancos de dados em nuvem são acessados pela internet, uma conexão estável e rápida é necessária. Qualquer interrupção na conectividade pode impactar o acesso aos dados.
- **Preocupações com a Segurança:** Embora os provedores de nuvem ofereçam altos níveis de segurança, algumas organizações podem ter preocupações em relação à privacidade e ao controle dos dados, especialmente em setores altamente regulamentados.
- **Latência:** O tempo de resposta pode ser maior comparado a sistemas locais, especialmente se os data centers do provedor de nuvem estiverem localizados longe dos usuários finais.

Nuvem (cloud)

- **Cenário de Funcionamento**

- Imagine uma empresa de desenvolvimento de software que utiliza um banco de dados em nuvem para armazenar dados de clientes e transações:
- **Desenvolvimento Ágil:** A empresa pode rapidamente criar novos ambientes de banco de dados para desenvolvimento e testes sem precisar configurar hardware físico.
- **Escalabilidade:** Se o número de usuários aumentar rapidamente após o lançamento de um novo produto, o banco de dados em nuvem pode automaticamente aumentar sua capacidade para lidar com a demanda.
- **Custo-Efetividade:** A empresa paga apenas pelos recursos utilizados, permitindo uma gestão eficiente do orçamento.
- A arquitetura em nuvem tem transformado a maneira como as empresas gerenciam seus dados, oferecendo uma combinação poderosa de flexibilidade, escalabilidade e custo-efetividade. Ela permite que as empresas concentrem seus recursos em inovação e crescimento, em vez de em infraestrutura e manutenção.

Nuvem (cloud)



Curiosidade

📌 Observação sobre Data Centers no Brasil

O Brasil possui aproximadamente 180 data centers, que são grandes estruturas físicas com milhares de servidores.

Eles armazenam e processam dados localmente, garantindo mais velocidade, segurança e redução de custos.

Muitos dados globais (como Netflix, YouTube, Instagram e Spotify) são replicados em data centers brasileiros, evitando que o tráfego precise ir até os EUA ou Europa.

Além disso, dados sensíveis (bancos, governo, saúde) ficam armazenados no país por exigência legal.

Resultado: os usuários acessam serviços digitais com mais rapidez, estabilidade e confiabilidade.

Referencias Bibliográficas

- **"Database System Concepts"** - Abraham Silberschatz, Henry F. Korth, S. Sudarshan
- **"Distributed Systems: Principles and Paradigms"** - Andrew S. Tanenbaum, Maarten Van Steen
- **"Designing Data-Intensive Applications"** - Martin Kleppmann
- **"Cloud Computing: Concepts, Technology & Architecture"** - Thomas Erl, Ricardo Puttini, Zaigham Mahmood
- **"Fundamentals of Database Systems"** - Ramez Elmasri, Shamkant B. Navathe